

WAFT

THEONE BEING IMPLEMENTATION PLAN

PLAN-THEONE-001

PLAN TYPE:	Implementation
COMPONENT:	Being System
PRIORITY:	High

IMPLEMENTATION PLAN

This document contains the complete implementation plan for creating TheOne as the root ancestor for all Beings.

Plan Details

name: Create TheOne Being

overview: Create "TheOne" as a special Being entity that serves as the root ancestor for all Beings.
todos:

- id: "1"
content: Add THE_ONE_BEING_ID constant and get_or_create_the_one() method to BeingSystem
status: pending
- id: "2"
content: Modify spawn_being() to ensure all Beings are descendants of TheOne
status: pending
- id: "3"
content: Add purpose-related attributes (purpose_being_id, purpose) to Being class
status: pending
- id: "4"
content: Add get_purpose(), set_purpose(), and imbue_with_purpose() methods to Being
status: pending
- id: "5"
content: Update Being.to_dict() and from_dict() to include purpose fields
status: pending
- id: "6"
content: Create genesis_reality for TheOne (or use existing reality system)
status: pending
- id: "7"
content: Create /pilot command file with execution steps
status: pending
- id: "8"
content: Test TheOne creation and verify ancestral chains include TheOne
status: pending

Create TheOne Being

Overview

Create "TheOne" as a special Being entity that serves as the root ancestor for all Beings. All

Architecture

Current State

- Beings have `source\_id = "source\_consciousness"` (abstract source)
- Ancestral chains: `[source\_consciousness, being\_id]`
- No physical Being entity at the root

Target State

- TheOne is a Being entity stored in `\_hidden/.truth/beings/the\_one.json`
- All new Beings have TheOne in their ancestral chain
- Ancestral chains: `[source\_consciousness, the\_one\_being\_id, being\_id]`
- TheOne is created once and loaded when needed

Implementation Steps

Step 1: Add TheOne Constants and Methods to BeingSystem

****File**:** `src/waft/being.py`

****Changes**:**

1. Add constant: `THE\_ONE\_BEING\_ID = "the\_one"`
2. Add method: `get\_or\_create\_the\_one() -> Being`:
 - Check if TheOne exists in beings directory
 - If exists, load and return
 - If not, create TheOne Being:
 - `being\_id = "the\_one"`
 - `reality\_id = "genesis\_reality"` (special reality)
 - `parent\_being\_id = None` (spawns from Source)
 - `source\_id = "source\_consciousness"`
 - `lifetimes = 1` (first Being)
 - `custom\_name = "TheOne"`
 - `ancestral\_chain = [source\_consciousness, the\_one]`
 - Save TheOne to disk
 - Return TheOne Being

Step 2: Modify spawn_being() to Include TheOne

****File**:** `src/waft/being.py`

****Changes in `spawn\_being()` method**:**

1. Get or create TheOne at start of method
2. If `parent\_being\_id` is None (spawning from Source):
 - Set `parent\_being\_id = the\_one.being\_id` (spawn from TheOne instead of Source)
3. Build ancestral chain:
 - If has parent: `[source\_consciousness, the\_one\_being\_id, ...parent\_chain[1:], being\_id]`

- If no parent (shouldn't happen now): `[source\_consciousness, the\_one\_being\_id, being\_id]`
4. Ensure TheOne is always in ancestral chain (even if parent provided)

Step 3: Update Being.__init__() Ancestral Chain Logic

****File**:** `src/waft/being.py`

****Changes**:**

- When `parent\_being\_id` is None, ancestral chain should still include TheOne
- This will be handled by `spawn\_being()`, but ensure initialization doesn't break

Step 4: Create Genesis Reality for TheOne

****File**:** `src/waft/being.py` (in `get\_or\_create\_the\_one()`)

****Changes**:**

- Check if `genesis\_reality` exists (using RealitySystem)
- If not, create it:
 - Type: `RealityType.GENESIS` (or `LEARNING` if GENESIS doesn't exist)
 - Configuration: `{"special": true, "purpose": "genesis"}`
 - Source: `"source\_consciousness"`

Step 5: Add Being.get_purpose() Method

****File**:** `src/waft/being.py`

****Changes**:**

- Add `purpose\_being\_id: Optional[str] = None` to `\_\_init\_\_()`
- Add `purpose: Optional[Dict[str, Any]] = None` to `\_\_init\_\_()` (purpose object)
- Add method: `get\_purpose() -> Optional[Dict[str, Any]]`:
 - Returns purpose dict if set
 - If `purpose\_being\_id` is set, load Purpose Being and return its purpose
 - Returns None if no purpose
- Add method: `set\_purpose(purpose: Dict[str, Any]) -> None`:
 - Sets purpose directly
- Add method: `imbue\_with\_purpose(purpose\_being: Being) -> None`:
 - Sets `purpose\_being\_id = purpose\_being.being\_id`
 - Links this Being to Purpose Being

Step 6: Update Being.to_dict() and from_dict()

****File**:** `src/waft/being.py`

****Changes**:**

- Add `purpose\_being\_id` and `purpose` to `to\_dict()`

- Add `purpose\_being\_id` and `purpose` to `from\_dict()`

Step 7: Create /pilot Command

****File**:** `.cursor/commands/pilot.md`

****Structure**:**

1. Parse context from user input (everything after `/pilot`)
2. Spawn new Being (will automatically be descendant of TheOne)
3. Create Purpose Being:
 - Spawn new Being with purpose derived from context
 - Purpose Being's purpose = parsed context/query
4. Imbue main Being with Purpose:
 - Call `being.imbue\_with\_purpose(purpose\_being)`
5. Have Being "live" and execute:
 - Set Being state to `LEARNING`
 - Being makes decisions to accomplish purpose
 - Being uses tools (codebase_search, file operations, etc.) to fulfill purpose
 - Being learns skills as it works
6. Save Being progress periodically

Files to Modify

1. `src/waft/being.py`:
 - Add `THE\_ONE\_BEING\_ID` constant
 - Add `get\_or\_create\_the\_one()` method
 - Modify `spawn\_being()` to use TheOne
 - Add purpose-related attributes and methods
 - Update `to\_dict()` and `from\_dict()`
2. `.cursor/commands/pilot.md`:
 - Create new command file following command template

Testing Considerations

- Verify TheOne is created on first spawn
- Verify TheOne is loaded on subsequent spawns
- Verify all new Beings have TheOne in ancestral chain
- Verify Purpose Being can be created and linked
- Verify Being can query its purpose

Migration Notes

- Existing Beings will not have TheOne in their chain (backward compatible)

- New Beings will automatically include TheOne
- TheOne is created lazily (on first spawn or when explicitly requested)

CURRENT SESSION CONTEXT

CURRENT TASK

Create TheOne as root ancestor for all Beings

Recent Topics

- Being System
- Ancestral Chains
- Purpose System
- /pilot Command

Key Decisions

- TheOne is a Being entity
- All Beings descend from TheOne
- Purpose is a Being that can evolve
- Lazy creation of TheOne

Next Steps

1. Implement get_or_create_the_one()
2. Modify spawn_being()
3. Add purpose methods
4. Create /pilot command

SYSTEM STATUS

SYSTEM STATUS REPORT

Report Date: 2026-01-13T09:59:40.803859

GIT REPOSITORY STATUS

Branch Information

Current Branch: feature/pdf-black-bar-fix-proof-system-20260113

Commits Ahead: 0

Commits Behind: 0

Uncommitted Changes

Total Uncommitted Files: 36

Staged Files: 28

Unstaged Files: 35

 **WARNING**

Large number of uncommitted files detected. Consider committing changes.

Recent Commits

- 7cad6ca feat: Remove black bars from PDF headers and add comprehensive proof system
- 68541db chore: bump version to 0.7.1
- 8480737 chore: bump version to 0.7.0
- c27fd71 feat: Create FlexiblePDFGenerator for evolving formatting ideas
- 48ae62f docs: Add cognitive processing entry to devlog

WORK EFFORTS STATUS

Work Efforts Breakdown

Category	Count
Total Work Efforts	58
Active	0
Recent	58

PROJECT HEALTH METRICS

Health Check Results

Component	Status	Details
_pyrite Structure	✔ Valid	Structure intact
Directory Structure	✔ Valid	active/ and backlog/ exist
Dependency Lock	✔ Present	uv.lock file exists

RECENT ACTIVITY

Devlog Entries: 1 recent entries

ANALYSIS

STATUS SUMMARY

- Git repository is synchronized
- Project structure is healthy
- Work effort activity: 0 active efforts
- Uncommitted changes: 36 files